

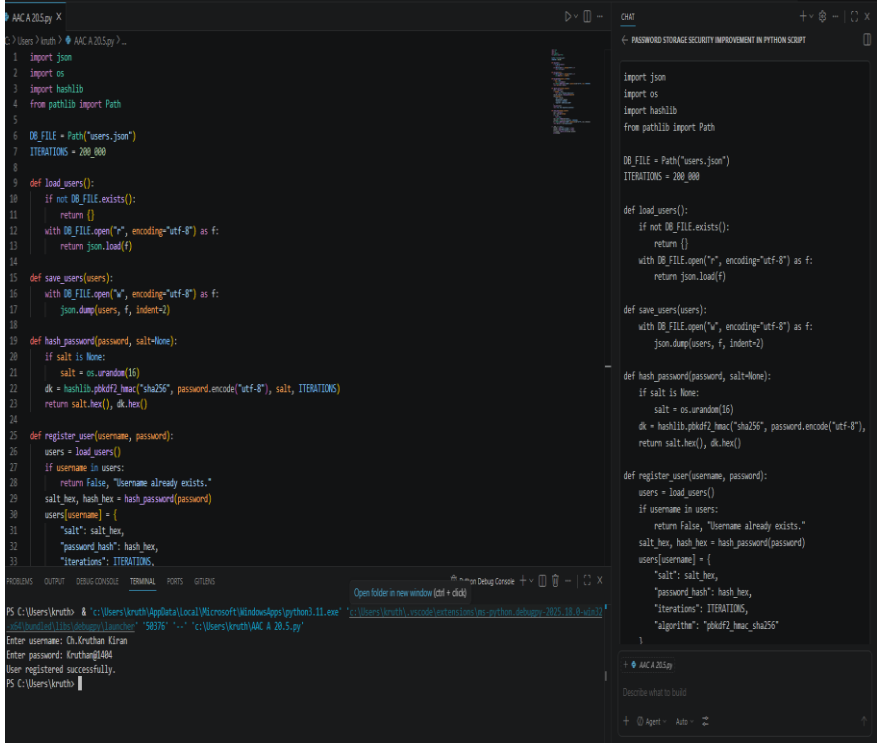
Name: J Mukhesh

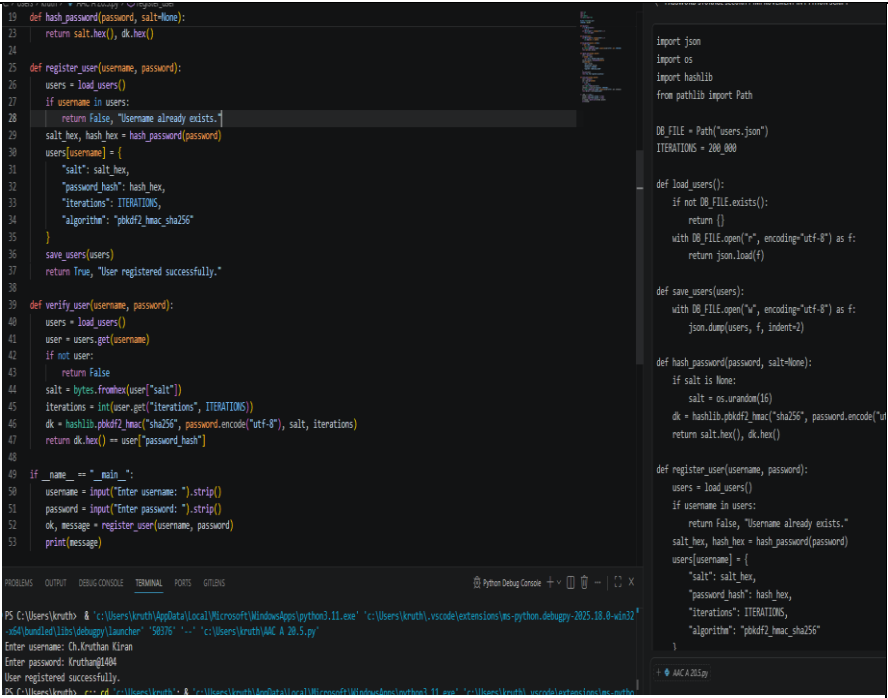
H.NO:2303A51813

BATCH:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week10 – Friday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number:20.5(Present assignment number)/24(Total number of assignments)																				

Q.No.	Question	Expected Time to complete
1	Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code	Week10 - Friday

	Lab Objectives: - Understand how to test AI-generated code for common security vulnerabilities. - Learn to apply secure coding principles while analyzing AI outputs. - Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption. - Enhance code reliability and safety by using AI for secure refactoring.	
	Task 1 – Password Storage Vulnerability Check Task: Analyze an AI-generated signup script for insecure password storage. Instructions: - Prompt AI to generate a Python user registration program. - Check whether passwords are stored in plain text. - Improve the script using hashing (bcrypt or hashlib). Expected Output: - A secure registration script with hashed password storage. 	

	 <pre>19 def hash_password(password, salt=None): 20 return salt.hex(), dk.hex() 21 22 23 24 25 def register_user(username, password): 26 users = load_users() 27 if username in users: 28 return False, "Username already exists." 29 salt_hex, hash_hex = hash_password(password) 30 users[username] = { 31 "salt": salt_hex, 32 "password_hash": hash_hex, 33 "iterations": ITERATIONS, 34 "algorithm": "pbkdf2_hmac_sha256" 35 } 36 save_users(users) 37 return True, "User registered successfully." 38 39 40 def verify_user(username, password): 41 users = load_users() 42 user = users.get(username) 43 if not user: 44 return False 45 salt = bytes.fromhex(user["salt"]) 46 iterations = int(user.get("iterations", ITERATIONS)) 47 dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations) 48 return dk.hex() == user["password_hash"] 49 50 if __name__ == "__main__": 51 username = input("Enter username: ").strip() 52 password = input("Enter password: ").strip() 53 ok, message = register_user(username, password) 54 print(message) 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99</pre> <pre>import json import os import hashlib from pathlib import Path DB_FILE = Path("users.json") ITERATIONS = 200_000 def load_users(): if not DB_FILE.exists(): return [] with DB_FILE.open("r", encoding="utf-8") as f: return json.load(f) def save_users(users): with DB_FILE.open("w", encoding="utf-8") as f: json.dump(users, f, indent=2) def hash_password(password, salt=None): if salt is None: salt = os.urandom(16) dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, ITERATIONS) return salt.hex(), dk.hex() def register_user(username, password): users = load_users() if username in users: return False, "Username already exists." salt_hex, hash_hex = hash_password(password) users[username] = { "salt": salt_hex, "password_hash": hash_hex, "iterations": ITERATIONS, "algorithm": "pbkdf2_hmac_sha256" } save_users(users) return True, "User registered successfully." def verify_user(username, password): users = load_users() user = users.get(username) if not user: return False salt = bytes.fromhex(user["salt"]) iterations = int(user.get("iterations", ITERATIONS)) dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, iterations) return dk.hex() == user["password_hash"] if __name__ == "__main__": username = input("Enter username: ").strip() password = input("Enter password: ").strip() ok, message = register_user(username, password) print(message)</pre> <pre>PS C:\Users\kruth> & "c:\Users\kruth\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher" "58376" "-." "c:\Users\kruth\AAC A 20.5.py" Enter username: Ch.Kruthan Kiran Enter password: Kruthan@1484 User registered successfully. PS C:\Users\kruth></pre>	
	Task 2 – Hardcoded Secrets Detection Task: Evaluate AI-generated code for hardcoded API keys or credentials. Instructions: • Ask AI to generate a script that connects to an external API. • Identify if API keys/passwords are written directly in code. • Refactor using environment variables or config files. Expected Output: • A secure script with secrets managed properly.	

```
AKA205.py X
C:\Users\kruth> AKA205.py get_weather
1 import os
2 import json
3 from pathlib import Path
4 import requests
5
6 BASE_URL = os.getenv("WEATHER_API_BASE_URL", "https://api.openweathermap.org/data/2.5/weather")
7 CONFIG_PATH = Path(__file__).with_name("weather_config.json")
8
9 def load_api_key() -> str | None:
10     env_key = os.getenv("WEATHER_API_KEY")
11     if env_key:
12         return env_key
13     if CONFIG_PATH.exists():
14         with CONFIG_PATH.open("r", encoding="utf-8") as f:
15             data = json.load(f)
16             key = data.get("WEATHER_API_KEY")
17             if key:
18                 return key
19     return None
20
21 def get_weather(city: str) -> dict:
22     api_key = load_api_key()
23     if not api_key:
24         api_key = input("Enter WEATHER_API_KEY: ").strip()
25     if not api_key:
26         raise RuntimeError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
27     params = {"q": city, "appid": api_key, "units": "metric"}
28     response = requests.get(BASE_URL, params=params, timeout=15)
29     if response.status_code == 401:
30         raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
31     response.raise_for_status()
32     return response.json()
33
34 if __name__ == "__main__":
35     try:
36         city = input("City: ").strip()
37         data = get_weather(city)
38         print(data)
39     except requests.exceptions.RequestException as e:
40         print(f"Network/API error: {e}")
41     except RuntimeError as e:
42         print(e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55964' '-' 'c:\Users\kruth\AKA 20.5.py'

raise HTTPError(http_error_msg, response=self)

requests.exceptions.HTTPError: 401 Client Error: Unauthorized for url: https://api.openweathermap.org/data/2.5/weather?q=Parkal&appid=84f61c6269619ae6348e093b0aa4209a&units=metric

PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50806' '-' 'c:\Users\kruth\AKA 20.5.py'

City: Parkal

Enter WEATHER_API_KEY: 84f61c6269619ae6348e093b0aa4209a

Invalid WEATHER_API_KEY. Please verify or generate a new key.

```
AAC A 20.5.py X
C:\Users\kruth> AAC A 20.5.py get_weather
21 def get_weather(city: str) -> dict:
22     api_key = load_api_key()
23     if not api_key:
24         api_key = input("Enter WEATHER_API_KEY: ").strip()
25     if not api_key:
26         raise RuntimeError("Missing WEATHER_API_KEY in environment, weather_config.json, and runtime input")
27     params = {"q": city, "appid": api_key, "units": "metric"}
28     response = requests.get(BASE_URL, params=params, timeout=15)
29     if response.status_code == 401:
30         raise RuntimeError("Invalid WEATHER_API_KEY. Please verify or generate a new key.")
31     response.raise_for_status()
32     return response.json()
33
34 if __name__ == "__main__":
35     try:
36         city = input("City: ").strip()
37         data = get_weather(city)
38         print(data)
39     except requests.exceptions.RequestException as e:
40         print(f"Network/API error: {e}")
41     except RuntimeError as e:
42         print(e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55964' '-' 'c:\Users\kruth\AAC A 20.5.py'

raise HTTPError(http_error_msg, response=self)

requests.exceptions.HTTPError: 401 Client Error: Unauthorized for url: https://api.openweathermap.org/data/2.5/weather?q=Parkal&appid=84f61c6269619ae6348e093b0aa4209a&units=metric

PS C:\Users\kruth> cd 'c:\Users\kruth'; & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50806' '-' 'c:\Users\kruth\AAC A 20.5.py'

City: Parkal

Enter WEATHER_API_KEY: 84f61c6269619ae6348e093b0aa4209a

Invalid WEATHER_API_KEY. Please verify or generate a new key.

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.

- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

```

AAC A 20.5.py
1 from flask import Flask, request, jsonify
2 from werkzeug.utils import secure_filename
3 from pathlib import Path
4 import uuid
5
6 app = Flask(__name__)
7 app.config['MAX_CONTENT_LENGTH'] = 5 * 1024 * 1024
8
9 UPLOAD_DIR = Path("uploads")
10 UPLOAD_DIR.mkdir(exist_ok=True)
11
12 ALLOWED_EXTENSIONS = ("png", "jpg", "jpeg", "pdf", "txt")
13 EXTENSION_TO_KIND = {
14     "png": ("img"),
15     "jpg": ("img"),
16     "jpeg": ("img"),
17     "pdf": ("pdf"),
18     "txt": ("txt"),
19 }
20
21 def detect_kind(data: bytes) -> str:
22     if data.startswith(b"\xff\xd9\xff"):
23         return "img"
24     if data.startswith(b"\xff\xff\xff"):
25         return "pdf"
26     if data.startswith(b"SGR-"):
27         return "pdf"
28     if data.startswith(b"%PDF"):
29         return "pdf"
30     if data.startswith(b"%PDF-1.4"):
31         return "pdf"
32     if b"PNG" in data:
33         return "img"
34     return "unknown"
35
36 @app.post("/upload")
37 def upload_file():
38     if "file" not in request.files:
39         return jsonify({"error": "No file part"}), 400
40
41     file = request.files["file"]
42     if file.filename is None or file.filename.strip() == "":
43         return jsonify({"error": "No selected file"}), 400
44
45     original_name = secure_filename(file.filename)
46     if original_name == "":
47         return jsonify({"error": "Invalid filename"}), 400
48
49     ext = Path(original_name).suffix.lower().lstrip(".")
50     if ext not in ALLOWED_EXTENSIONS:
51         return jsonify({"error": "File extension not allowed"}), 400
52
53     head = file.stream.read(8192)
54     if not head:
55         return jsonify({"error": "Empty file"}), 400
56
57     kind = detect_kind(head)
58     if kind in ("exe", "zip", "binary", "unknown"):
59         return jsonify({"error": "Unsafe or unsupported file content"}), 400
60
61     if kind not in EXTENSION_TO_KIND[ext]:
62         return jsonify({"error": "File extension not allowed"}), 400
63
64     filename = f"{uuid.uuid4()}.{ext}"
65     file.save(UPLOAD_DIR / filename)
66     return jsonify({"message": "File uploaded successfully"}), 200
67
68 if __name__ == "__main__":
69     app.run(debug=True)
70
71 PASSWORD STORAGE SECURITY IMPROVEMENT IN PYTHON SCRIPT
72
73 from flask import Flask, request, jsonify
74 from werkzeug.utils import secure_filename
75 from pathlib import Path
76 import uuid
77
78 app = Flask(__name__)
79 app.config['MAX_CONTENT_LENGTH'] = 5 * 1024 * 1024
80
81 UPLOAD_DIR = Path("uploads")
82 UPLOAD_DIR.mkdir(exist_ok=True)
83
84 ALLOWED_EXTENSIONS = ("png", "jpg", "jpeg", "pdf", "txt")
85 EXTENSION_TO_KIND = {
86     "png": ("img"),
87     "jpg": ("img"),
88     "jpeg": ("img"),
89     "pdf": ("pdf"),
90     "txt": ("txt"),
91 }
92
93 def detect_kind(data: bytes) -> str:
94     if data.startswith(b"\xff\xd9\xff"):
95         return "img"
96     if data.startswith(b"\xff\xff\xff"):
97         return "pdf"
98     if data.startswith(b"SGR-"):
99         return "pdf"
100     if data.startswith(b"%PDF"):
101         return "pdf"
102     if data.startswith(b"%PDF-1.4"):
103         return "pdf"
104     if b"PNG" in data:
105         return "img"
106     return "unknown"
107
108 @app.post("/upload")
109 def upload_file():
110     if "file" not in request.files:
111         return jsonify({"error": "No file part"}), 400
112
113     file = request.files["file"]
114     if file.filename is None or file.filename.strip() == "":
115         return jsonify({"error": "No selected file"}), 400
116
117     original_name = secure_filename(file.filename)
118     if original_name == "":
119         return jsonify({"error": "Invalid filename"}), 400
120
121     ext = Path(original_name).suffix.lower().lstrip(".")
122     if ext not in ALLOWED_EXTENSIONS:
123         return jsonify({"error": "File extension not allowed"}), 400
124
125     head = file.stream.read(8192)
126     if not head:
127         return jsonify({"error": "Empty file"}), 400
128
129     kind = detect_kind(head)
130     if kind in ("exe", "zip", "binary", "unknown"):
131         return jsonify({"error": "Unsafe or unsupported file content"}), 400
132
133     if kind not in EXTENSION_TO_KIND[ext]:
134         return jsonify({"error": "File extension not allowed"}), 400
135
136     filename = f"{uuid.uuid4()}.{ext}"
137     file.save(UPLOAD_DIR / filename)
138     return jsonify({"message": "File uploaded successfully"}), 200
139
140 if __name__ == "__main__":
141     app.run(debug=True)
142
Terminal
C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-ebugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57474' '-.' 'c:\Users\kruth\AAC A 20.5.py'
ModuleNotFoundError: No module named 'flask'
PS C:\Users\kruth> cd 'c:\Users\kruth' & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-ebugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57510' '-.' 'c:\Users\kruth\AAC A 20.5.py'
* Serving Flask app 'AAC A 20.5'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit

```



```
AAC A 20.5.py X
C:\Users\kruth> AAC A 20.5.py & run_safe

24 def run_safe(command: str, target: str, count: int = 4) -> str:
25     cmd = command.strip().lower()
26     if cmd not in ALLOWED_COMMANDS:
27         raise ValueError("Unsupported command")
28     safe_target = validate_target(target)
29     safe_count = validate_count(str(count))
30     if cmd == "ping":
31         args = ["ping", "-n", str(safe_count), safe_target]
32     else:
33         args = ["nslookup", safe_target]
34     result = subprocess.run(args, capture_output=True, text=True, timeout=15, shell=False, check=False)
35     output = result.stdout if result.stdout else result.stderr
36     return output.strip()
37
38 if __name__ == "__main__":
39     user_command = input("Command (ping/nslookup): ")
40     user_target = input("Target (domain or host): ")
41     user_count = input("Ping count (1-10, default 4): ").strip() or "4"
42     try:
43         print(run_safe(user_command, user_target, int(user_count)))
44     except Exception as e:
45         print(f"Error: {e}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\kruth> & 'c:\Users\kruth\anaconda3\envs\kiran\python.exe' 'c:\Users\kruth\.vscode\extensions\ms-python.
ebuppy\launcher' '64924' '--' 'c:\Users\kruth\AAC A 20.5.py'
Reply from 142.250.182.78: bytes=32 time=22ms TTL=116
Reply from 142.250.182.78: bytes=32 time=21ms TTL=116
Reply from 142.250.182.78: bytes=32 time=22ms TTL=116

Ping statistics for 142.250.182.78:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 21ms, Maximum = 130ms, Average = 48ms
```

Task 5 – Secure Input Handling in Web Forms

Task:

Check AI-generated web form code for improper input validation.

Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

- A secure form resistant to injection attacks.

```
WGA205.py X
C:\Users\kruth> AAC A 20.5.py
1 from flask import Flask, request, render_template_string, redirect, url_for, session, abort
2 import re
3 import secrets
4
5 app = Flask(__name__)
6 app.config["SECRET_KEY"] = secrets.token_hex(32)
7 app.config["MAX_CONTENT_LENGTH"] = 16 * 1024
8
9 EMAIL_RE = re.compile(r"^[a-zA-Z0-9.-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$")
10 NAME_RE = re.compile(r"^[a-zA-Z0-9- ]+$")
11
12 feedback_store = []
13
14 PAGE = """
15 <doctype html>
16 <html lang=en>
17 <head>
18 <meta charset=utf-8>
19 <meta name=viewport content=device-width,initial-scale=1>
20 <title>Secure Feedback Form/Title
21 </title>
22 <body {font-family: Segoe UI, sans-serif; max-width: 768px; margin: 10px auto; padding: 0 10px;}
23 <form {display: grid; gap: 10px;}
24 input, textarea {width: 100%; padding: 5px;}
25 button {padding: 5px 10px; width: max-content;}
26 <div {color: #000080; margin-top: 10px;}
27 <div {color: #000080; margin-top: 10px;}
28 <div {padding-left: 10px;}
29 </div>
30 </body>
31 </html>
32
33 <div>feedbacks</div>
34
35 """
36
37 @app.route("/")
38 def index():
39     return render_template_string(PAGE, feedback_store=feedback_store)
40
41 @app.route("/submit", methods=["POST"])
42 def submit():
43     name = request.form.get("name")
44     email = request.form.get("email")
45     feedback = request.form.get("feedback")
46
47     if not NAME_RE.match(name) or not EMAIL_RE.match(email):
48         return render_template_string(PAGE, feedback_store=feedback_store, error="Invalid email or name")
49
50     feedback_store.append({"name": name, "email": email, "feedback": feedback})
51     return render_template_string(PAGE, feedback_store=feedback_store)
52
53 if __name__ == "__main__":
54     app.run(debug=True)
```

```
MCA205py 1 X
1 from flask import Flask, request, render_template_string, redirect, url_for, session, abort
2 import re
3 import secrets
4
5 app = Flask(__name__)
6 app.config["SECRET_KEY"] = secrets.token_hex(32)
7 app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///"
8
9 DB_URL_RE = re.compile(r"^(?:[a-z0-9]+(?:[a-z0-9-]+)?(?:[a-z0-9]+\.){1,63}[a-z0-9]+)$")
10 NAME_RE = re.compile(r"^[a-z0-9-]{1,100}$")
11
12 feedback_store = []
13
14 page = """
15 <!doctype html>
16 <html lang=en>
17 <head>
18 <meta charset="utf-8">
19 <meta name="viewport" content="width=device-width,initial-scale=1">
20 <title>Secure Feedback Form</title>
21 <style>
22 body { font-family: Segoe UI, sans serif; max-width: 768px; margin: 32px auto; padding: 0 16px; }
23 form { display: grid; gap: 16px; }
24 input, textarea { width: 100%; padding: 16px; }
25 button { padding: 16px 16px; width: 100%; text-align: center; }
26 <div class="error">{{ error }}</div>
27 <div class="success">{{ success }}</div>
28 <div class="message">{{ message }}</div>
29 </style>
30 </head>
31 <body>
32 <div class="header">
33 <h1>Secure Feedback Form</h1>
34 </div>
35 <div class="form">
36 <div class="error">{{ error }}</div>
37 <div class="success">{{ success }}</div>
38 <div class="message">{{ message }}</div>
39 <div class="input">
40 <input type="text" name="name" value="{{ name }}" />
41 <input type="text" name="email" value="{{ email }}" />
42 <input type="text" name="message" value="{{ message }}" />
43 </div>
44 <div class="button">
45 <button type="submit">Submit</button>
46 </div>
47 </div>
48 </body>
49 </html>
50 """
51
52 @app.route("/")
53 def index():
54     return render_template_string(page)
55
56 @app.route("/feedback")
57 def feedback():
58     name = request.form.get("name")
59     email = request.form.get("email")
60     message = request.form.get("message")
61
62     if not name or not email or not message:
63         return render_template_string(page, error="Name, email, and message are required.")
64
65     if not DB_URL_RE.match(name):
66         return render_template_string(page, error="Name is invalid.")
67
68     if not NAME_RE.match(email):
69         return render_template_string(page, error="Email is invalid.")
70
71     feedback_store.append({"name": name, "email": email, "message": message})
72
73     return render_template_string(page, success="Feedback submitted successfully.", message="Hello There its Kruthan (2303A51404)")
74
75 if __name__ == "__main__":
76     app.run(host="0.0.0.0", port=5000)
```

AAC A.20.5.py 1 Secure Feedback Form X

→ http://127.0.0.1:5000/feedback

Feedback

Name

Email

Message

Submit

Recent Feedback

- No feedback yet.

→ http://127.0.0.1:5000/feedback

Feedback

Feedback submitted successfully.

Name

Email

Message

Submit

Recent Feedback

- Kruthan Kiran** (kruthankiran@gmail.com): Hello There its Kruthan (2303A51404)

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.